

CONTEXTGRAPH: REMEMBER THE CONDITION, NOT THE CODE

Anonymous authors

Paper under double-blind review

ABSTRACT

Coding agents can repeat costly debugging mistakes even when useful lessons exist in other repositories. Reusing this experience requires preserving transferable decisions, connecting related lessons, and retrieving them when they can guide the next action. We propose ContextGraph, an experience graph for self-improving coding agents. It distills strategies and warnings together with their applicability conditions and intended outcomes, retaining links to the source experience. The agent selects guidance using its full context before planning and searches again after errors to revise its diagnosis. After each task, successful and failed trajectories contribute new experience that subsequent tasks can immediately retrieve. ContextGraph achieves resolution rates of 69.2%, 35.1%, and 72.1% on SWE-bench Verified, SWE-ContextBench Related-Lite, and Datacurve DeepSWE, improving over no memory by 7.6, 4.5, and 11.3 percentage points, respectively. It leads the evaluated methods on Verified and DeepSWE. Behavioral studies expose negative transfer from implementation reuse and show how composing historical conditions with current operations catches failures missed by separate checks.

1 INTRODUCTION

Each repair produces experience in locating faults, revising code, and checking behavior. Memory reuses this experience on later tasks, offering a practical route toward recursive self-improvement (RSI) (Yin et al., 2025; Zhang et al., 2026a). Both successful and failed attempts contribute.

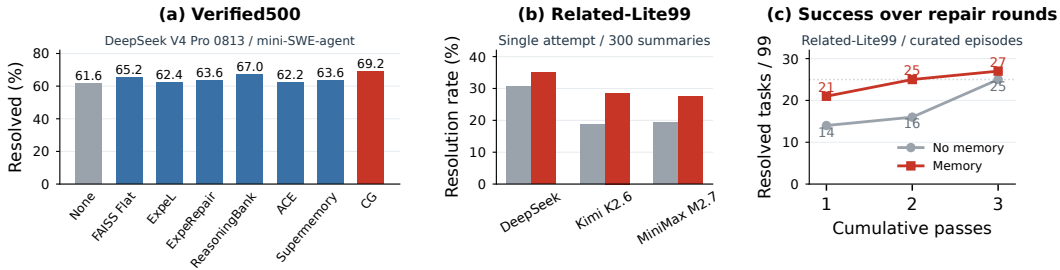


Figure 1: ContextGraph improves repair success across the evaluated settings. (a) SWE-bench Verified500 with DeepSeek V4 Pro 0813 and mini-SWE-agent; (b) Related-Lite99 with three language models; (c) cumulative resolutions on Related-Lite99 with DeepSeek V4 Pro. Grey denotes no memory and red ContextGraph.

Recent agent-memory methods retain reflections and repair lessons (Shinn et al., 2023; Zhao et al., 2024) and organize this experience for reuse (Ouyang et al., 2026; Zhang et al., 2026b). To transfer across repositories, a lesson must guide investigation in unfamiliar code and identify approaches to avoid. The agent then determines the local change. Section 3 illustrates this transfer to Django from an earlier setup failure and a successful debugging attempt.

We therefore ask: *How can coding-agent memory preserve the conditions behind a useful decision and help an agent adapt that decision to unfamiliar code?* Our central idea is to remember when a lesson applies and what behavior it should preserve, so the agent can adapt and check it in the current task.

Building this memory involves three challenges. First, a lesson must retain the useful decision and its applicability conditions while abstracting away repository-specific names and code. Second, related experiences are scattered across repositories: similar words can describe different problems, while different errors can call for the same strategy. Retrieval needs these connections. Third, planning needs direction for investigation, while errors bring evidence that can change the diagnosis. Memory must inform both decisions, then retain the completed attempt for later tasks.

To address these challenges, we propose **ContextGraph**, a memory graph that turns experience across repositories into guidance for new repairs (Figure 3). *Experience abstraction* extracts strategies and warnings: what to do, what to avoid, and when each lesson applies. The graph links them to source experiences, problems, and errors. *Agent-directed selection* uses the full task context to search the experience pool and follow graph connections, selecting useful strategies across repositories. *Memory access* emphasizes two points: before planning and after an error. The first query uses the available task context; the second adds the error and recent attempt to guide diagnosis and revision. The agent applies these lessons through local edits and tests. After every task, ContextGraph distills the successful or failed trajectory and updates the graph, making new experience immediately available to subsequent tasks.

ContextGraph reaches 69.2% resolution on SWE-bench Verified and 72.1% on DeepSWE113, exceeding the strongest evaluated memory baselines by 2.2 and 5.9 percentage points, respectively. On Related-Lite99, its mean resolution rate is 35.1%, 4.5 points above no memory (Figure 4). Figure 1 previews the benchmark, model, and retry comparisons. On 20 Python tasks from LoLBench, ContextGraph reaches 15.0% pass@1 versus 5.0% for all baselines, and 25.0% pass@3.

The contributions are summarized as follows:

- We propose ContextGraph, a memory graph for reusing coding experience across repositories. It connects abstract strategies and warnings to the problems, errors, and outcomes that explain when the guidance is useful.
- We integrate memory retrieval into planning and error recovery. The full task context guides memory selection; observed failures guide subsequent searches. Successful and failed trajectories update the graph after each task, connecting current repair decisions to continuing experience accumulation.
- ContextGraph leads the evaluated methods on SWE-bench Verified, DeepSWE113, and the LoLBench Python subset, and improves Related-Lite99 by 4.5 points over no memory. Behavioral studies identify negative transfer from implementation reuse and show the benefit of checking historical conditions within current operations.

2 RELATED WORK

Abstracting experience for coding agents. ExpeL and ReasoningBank extract trajectory lessons (Zhao et al., 2024; Ouyang et al., 2026). SWE-Exp and ExpeRepair reuse repair experience; MemGovern and STAIR organize cards and plans (Chen et al., 2025; Mu et al., 2026; Wang et al., 2026; Xu et al., 2026). Memory Transfer Learning finds that abstract insights transfer across coding domains more readily than detailed traces (Kim et al., 2026). Detailed traces can instead carry misleading implementation assumptions. ContextGraph retains strategies and warnings with their applicability context and outcomes. RETester generates tests from bug reports, MIRA retains requirements, and TRACE turns corrections into runtime checks (Wang et al., 2025; Liu et al., 2026; Zhou et al., 2026), connecting experience to executable feedback.

Retrieving related experience. A-MEM links notes, Zep organizes temporal facts, and HippoRAG retrieves through graph traversal (Xu et al., 2025; Rasmussen et al., 2025; Gutiérrez et al., 2024). ExpGraph combines experience graphs, personalized PageRank, utility ranking, and a trained retrieval controller (Feng et al., 2026). ContextGraph connects repair strategies to recurring errors, shared rules, and source trajectories. The agent uses its full context to search these records and select useful guidance. Supermemory combines relational updates and hybrid retrieval (Daga et al., 2026); SWE-ContextBench evaluates memory-assisted repair (Zhu et al., 2026).

Updating memory and improving agents. MemGPT and SWE-MeM manage memory access and coding histories (Packer et al., 2023; Gao et al., 2026). ACE evolves context, MemOp optimizes memory utility, and AdaRepair-Mem selects experience by repair stage (Zhang et al., 2026b; Guo

et al., 2026; Luo et al., 2026). Gödel Agent and the Darwin Gödel Machine modify agent code; RRSI regularizes harness changes (Yin et al., 2025; Zhang et al., 2026a; Xia et al., 2026). AlphaEvolves evolves programs through executable evaluation; MulVul evolves prompts across generator and executor models (Novikov et al., 2025; Wu et al., 2026). ContextGraph adds lessons from successes and failures after each task and makes them immediately available for retrieval. Subsequent tasks retrieve this growing experience before planning and after errors, without updating model parameters.

3 PROBLEM AND MOTIVATION

A coding agent receives an issue and a repository and produces a patch. Its trajectory records actions, observations, and the outcome. ContextGraph builds an experience graph G from completed trajectories, retrieves guidance for each new task, and updates G afterward. The goal is to accumulate useful repair experience across tasks and repositories.

A debugging decision that transfers. In Django Trac #33018, empty query results are mishandled under negation. A failed `python-statemachine` attempt had spent its effort setting up a full Django application; its lesson was to isolate the failure before investing in framework setup. A successful `sievelib` trajectory also supported reproducing the failure before editing. Guided by these experiences, the agent traced the exception to `Exists.as_sql()`, corrected its handling, and added a regression test. The no-memory attempt made broad formatting changes without fixing the bug. The transferable content was the investigation strategy and warning; the agent derived the local implementation.

From advice to a concrete check. Remembered guidance also needs to be applied to the current operation. Figure 2 illustrates another Django case: a historical requirement becomes a check by combining its input condition with the current query. Section 5.5 compares separate and composed checks.

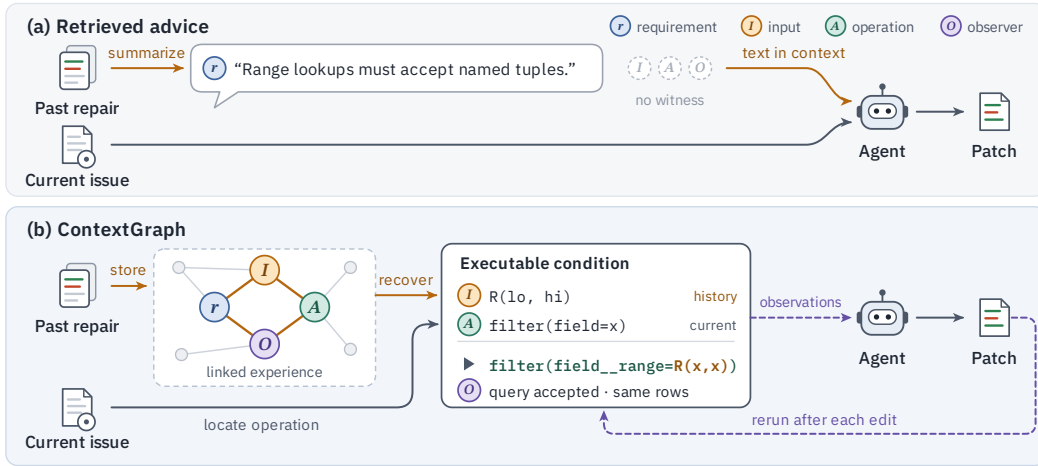


Figure 2: Django12663 behavioral study comparing (a) a requirement supplied as text with (b) linked context used to combine a historical range type with the current query and obtain execution feedback for the next edit.

4 CONTEXTGRAPH: MEMORY FOR PLANNING, RECOVERY, AND LEARNING

ContextGraph connects experience abstraction, agent-directed selection, and memory access. The same process builds the initial graph and incorporates completed tasks (Figure 3).

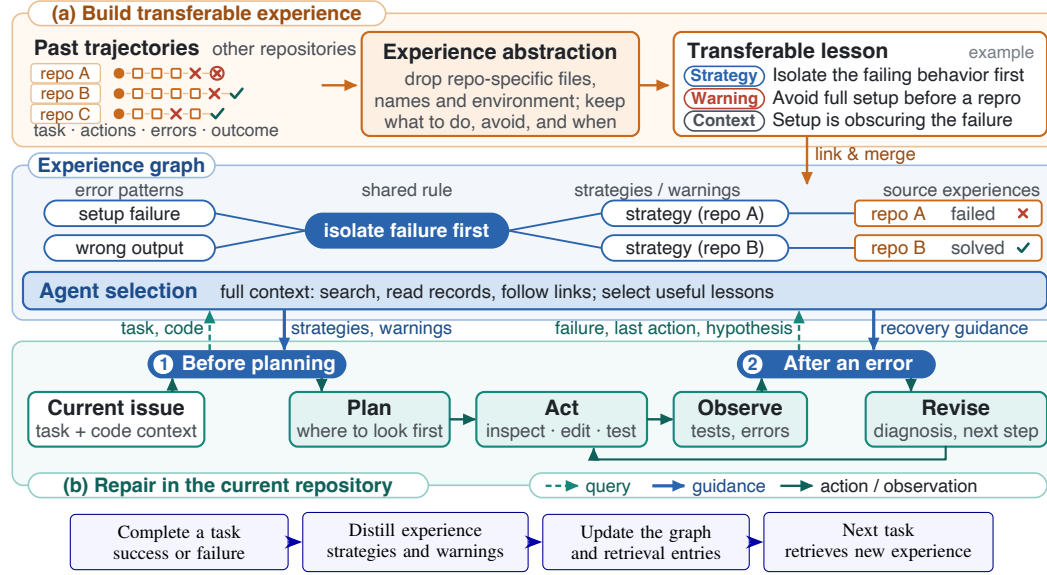


Figure 3: ContextGraph’s experience-memory architecture. Past trajectories supply strategies and warnings linked to source context. The agent retrieves guidance using its full context before planning and after errors. After every task, successful and failed trajectories update the graph; the added experience is immediately available to subsequent tasks. Graph links illustrate the organization.

4.1 EXPERIENCE ABSTRACTION: RETAIN THE USEFUL DECISION

A language model extracts lessons from the task, actions, errors, and outcome. Each lesson records an action or warning, its applicability conditions, and source context. Abstraction removes repository-specific names while retaining the decision and its reason: the failed framework setup becomes advice to isolate the bug first.

We represent memory as a typed graph

$$G = (V, E), \quad E \subseteq V \times \mathcal{R} \times V, \quad (1)$$

where each edge has a relation type in $\mathcal{R}$. Nodes represent trajectories, fragments (action–observation segments), strategies, canonical rules (merged lessons), error patterns, problem summaries, playbook entries (retrieval-ready lessons), and communities of related fragments. The seven edge types connect a trajectory to its fragments, a strategy to its source trajectory, a fragment to an encountered error, a strategy to its canonical rule, a rule to an error it addresses, a summary to its trajectory, and a fragment to its community. These links retain both a reusable lesson and the observations that explain when it applies.

4.2 AGENT-DIRECTED SELECTION: FIND USEFUL EXPERIENCE

At repair step t , the agent uses its full context C_t : the issue, inspected code, actions, and failures. Memory selection is an agent decision process:

$$a_j \sim \pi_\theta(\cdot \mid C_t, H_j), \quad \mathcal{M}_t = \text{Select}_{\pi_\theta}(C_t, G_t), \quad (2)$$

where π_θ is the coding agent’s model policy, H_j records its preceding memory actions and their results, and $\mathcal{M}_t$ contains the selected lessons. Actions search the entire pool, read source records, follow graph links, select lessons, or stop. The agent judges relevance from the returned context; there is no fixed numerical relevance score or top- k cutoff. It applies the selected guidance through local reproductions, edits, and tests (Appendix E).

4.3 MEMORY ACCESS AND UPDATES: CONNECT REPAIR TO LEARNING

Before planning. The full task context guides a search for investigation, reproduction, and testing strategies, together with warnings about costly approaches to avoid.

After an error. The failed action, observed error, relevant code, and current hypothesis update the context. The agent searches again to revise its diagnosis and uses repository tools to try the next step.

After each task. ContextGraph distills successful and failed trajectories and links new lessons to existing experience. Updates use agent-visible observations and become immediately available to later tasks (Appendix Algorithm 1).

5 EXPERIMENTS

We ask whether ContextGraph improves repair success, how its benefits vary across models and memory-pool sizes, whether memory reduces repeated repair effort, and how retrieved experience changes an agent’s actions.

5.1 SETUP AND EVALUATION

Tasks and metrics. We evaluate all 500 SWE-bench Verified tasks, the 99-task Related-Lite subset of SWE-ContextBench, and the 113-task Datacurve DeepSWE benchmark (Jimenez et al., 2024; Zhu et al., 2026; Huang et al., 2026). We additionally evaluate 20 Python tasks from LoLBench (lolbench26, 2026). A task is resolved when its patch passes the benchmark’s official evaluation. Resolution is the percentage of tasks resolved; $\text{pass}@k$ reports success within k attempts. Related-Lite99 and DeepSWE113 in the main comparison report means over five runs, each reset to the same initial memory pool. Table 1 gives the agents, models, and initial memory sources (SWE-agent Team, 2025; Yang et al., 2024).

Table 1: Main benchmark comparisons. The first three use DeepSeek V4 Pro 0813 with mini-SWE-agent; LoLBench uses GPT-5.5 with Codex. ContextGraph updates its memory after every task; baseline updates follow each implementation’s default behavior.

Benchmark	Tasks	Initial experience source	Reported result
Verified500	500	Nebius Verified trajectories	Resolution
Related-Lite99	99	Coding-agent trajectories	Five-run mean
DeepSWE113	113	LoLBench repository trajectories	Five-run mean
LoLBench Python	20	Other LoLBench Python tasks	$\text{pass}@1$, $\text{pass}@3$

Data separation and feedback. Initial memory excludes the evaluated tasks’ trajectories and reference solutions, and development tasks do not overlap the test sets. Tasks enter the evaluation stream in issue creation-time order. Memory updates use only agent-visible actions and observations, including its own test runs. Official test results are used for scoring only: they are neither shown to the agent nor written into memory, including between repeated attempts.

Baselines. We compare no memory, FAISS Flat, ExpeL, ExpeRepair, ReasoningBank, ACE, and Supermemory (Zhao et al., 2024; Mu et al., 2026; Ouyang et al., 2026; Zhang et al., 2026b; Daga et al., 2026). FAISS Flat uses vector similarity to retrieve memory without graph traversal. We adapt ExpeRepair and ACE to mini-SWE-agent, retaining their memory procedures. Supplementary model, pool-size, and retry comparisons state their own inputs and budgets alongside the results. We compare complete memory systems, including their construction, selection, and updates. Appendix Table 6 lists all main comparison values.

5.2 REPAIR EFFECTIVENESS

Figure 4 compares eight methods. ContextGraph reaches 69.2% on Verified500 and 72.1% on DeepSWE113, exceeding the strongest memory baselines, ReasoningBank and Supermemory, by 2.2 and 5.9 points. On Related-Lite99, its 35.1% mean improves on no memory and FAISS Flat by 4.5 and 8.3 points, respectively. **Finding:** ContextGraph improves repair success on all three benchmarks, leading the evaluated methods on Verified500 and DeepSWE113.

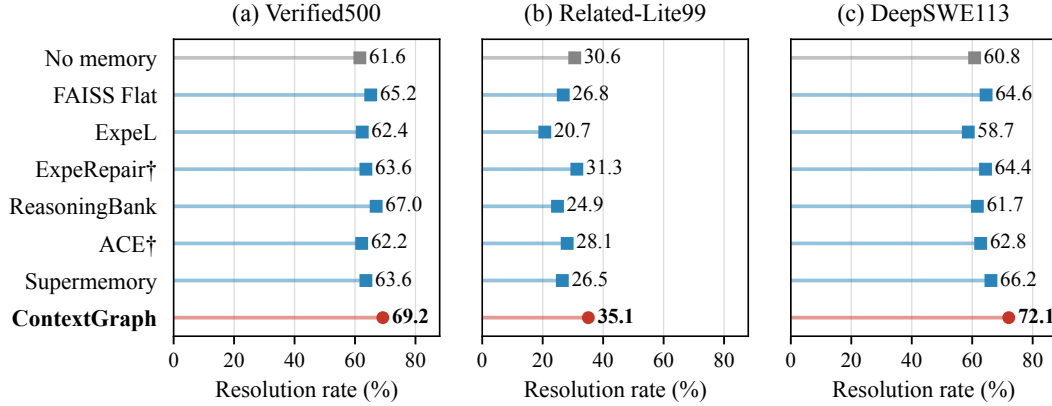


Figure 4: Repair resolution rates (%) with DeepSeek V4 Pro 0813 and mini-SWE-agent. Related-Lite99 and DeepSWE113 report five-run means. Red circles denote ContextGraph; bold values mark the best result per benchmark. † marks our adaptations to mini-SWE-agent.

Long-horizon development. With Codex and GPT-5.5, we build the graph from other Python tasks and evaluate 20 LoLBench tasks. All six memory baselines match no memory at 5.0% pass@1; ContextGraph reaches 15.0% pass@1 and 25.0% pass@3 (Appendix Table 12). **Finding:** ContextGraph achieves the highest pass@1 among the evaluated methods on this 20-task long-horizon subset.

Benefits with Gemini and live-SWE-agent. With Gemini 3.1 Pro and live-SWE-agent, ContextGraph raises Verified500 pass@1 from 82.3% to 88.9%, an 8.0% relative gain; pass@5 reaches 91.7% (Table 2). Memory also benefits this stronger backbone and a different coding-agent implementation.

Table 2: SWE-bench Verified500 with Gemini 3.1 Pro and live-SWE-agent. Both arms follow the data-separation protocol in Section 5.1. Gains are relative improvements, (ContextGraph – No memory)/No memory, in percent.

Metric	No memory	ContextGraph	Relative gain
pass@1	82.3%	88.9%	+8.0%
pass@3	82.6%	90.4%	+9.4%
pass@5	82.8%	91.7%	+10.7%

5.3 PERFORMANCE ACROSS MODELS AND MEMORY-POOL SIZES

Benefits across language models. In the Related-Lite99 single-attempt comparison, memory methods draw from the same 300-summary pool within each model (Figure 5), including Mem0 (Chhikara et al., 2025) and LangMem (LangChain, 2025). With DeepSeek V4 Pro, ContextGraph reaches 35.1% resolution, compared with 30.6% without memory. The oracle is given one known related summary, bypassing retrieval; ContextGraph can retrieve several. **Finding:** memory reuse improves resolution with all three models, with the highest non-oracle resolution rate in each model comparison.

Pool size and source coverage. With the DeepSeek V4 Pro agent and retrieval procedure fixed, the 300-summary pool produces the highest resolution rate, 35.1% (Table 3). Source coverage measures the proportion of oracle-designated relevant sources recovered by retrieval. It rises from 41.3% to 60.7% as the pool grows from 50 to 300 summaries, while resolution does not improve at every intermediate step. **Finding:** larger pools recover more oracle-designated sources, and the largest tested pool yields the strongest repair result.

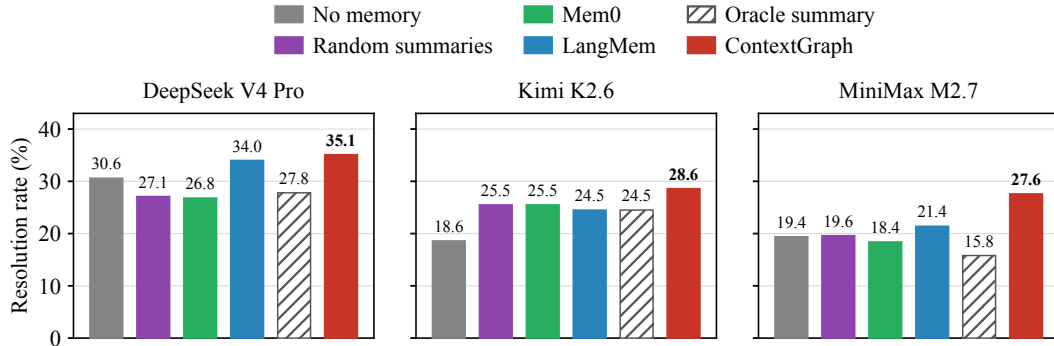


Figure 5: Related-Lite99 resolution (%), one attempt per task and 300 summaries per model. ContextGraph is red; the hatched oracle receives a known related summary.

Table 3: Related-Lite99 with different memory-pool sizes, DeepSeek V4 Pro, one attempt per task. Source coverage is the percentage of oracle-designated relevant sources recovered by retrieval.

Number of summaries	50	100	200	300
Resolution rate	21.88%	27.55%	26.80%	35.1%
Source coverage (%)	41.3	45.8	51.4	60.7

5.4 REPAIR SUCCESS WITH REPEATED ATTEMPTS

A separate Related-Lite99 experiment uses DeepSeek V4 Pro with 500 calls and \$30 per task in both conditions. The memory contains 291 curated repair records with reference fixes, verification hints, and history preceding the target code version. Each pass is a repair round; later passes retry only unresolved tasks. ContextGraph reaches 25 resolutions in two passes; the no-memory agent needs three (Table 4). Excluding ten targets with near-duplicate source fixes retains the first-pass gain, 19/89 versus 13/89 ($p = .031$). **Finding:** useful experience helps the agent reach successful repairs with fewer rounds, with the largest gains in the first two passes.

Additional system results appear in Appendices B and D.

5.5 WHAT SHOULD MEMORY PRESERVE AND HELP THE AGENT CHECK?

In the behavioral studies, we make this principle executable by combining a remembered input condition with the current operation and checking the intended behavior (Figure 2).

Condition composition. Let P_q be the current program with input condition c_q , and memory m supply a historical condition c_h and requirement r . A binding b maps historical input roles to the current program. A composed check is

$$W_{q,h} = \text{Compose}_b(P_q, m) = (I_{q,h}, A_q, O_r), \quad I_{q,h}() \models c_q \wedge b(c_h), \quad (3)$$

where $I_{q,h}$ constructs a valid joint input, A_q performs the current operation, and O_r checks the required behavior. In Django12663, this puts the current lazy User inside a historical named-tuple range. The studies use researcher-written range and timezone adapters (Appendix I).

Proposition 1 (detecting interaction failures). Fix a candidate repair and a correct requirement observer. Let $F(x)$ denote a violation and $C(x)$ the joint condition, with $F(x) \Rightarrow C(x)$. If every separate check $x \in S$ satisfies $\neg C(x)$, sampling from S has zero detection probability. A composed suite detects the violation if it contains a valid joint check x^* with $F(x^*)$. *Proof.* For each $x \in S$, $\neg C(x)$ implies $\neg F(x)$, so every separate check misses. The composed check x^* executes the violation, which the observer detects. Composition thus reaches interactions absent from separate checks; the joint condition alone need not cause a failure.

Table 4: Related-Lite99 with curated episodic memory. Resolved counts are cumulative. Wins count tasks solved only with memory; losses count tasks solved only without it. The exact McNemar p tests whether these paired outcomes differ.

Retry budget	No memory	ContextGraph	Wins/losses	p
1 pass	14/99	21/99	7/0	.016
2 passes	16/99	25/99	9/0	.0039
3 passes	25/99	27/99	6/4	.75

Preserve the requirement without inheriting an unsuitable implementation. In the timezone study, requirement-level memory passes all 72 combinations of database, application, and output zones. Reusing the historical implementation passes 40 and breaks 16 combinations that pass without memory. The historical implementation carries assumptions that fail in the new setting, causing negative transfer (Table 5).

Check historical conditions together with the current operation. For Django12663, three agents receive the same task, history, current program, tools, and repair budget. They receive current feedback alone, additional separate historical checks, or additional composed checks. All three repairs pass official verification. Both added-check conditions recover the two historical examples, but their joint behavior differs: separate checks pass only one of three withheld joint queries, while composed checks pass all three. The current-feedback repair passes two. These results illustrate the interaction addressed by Proposition 1; Appendix F gives the full studies. **Finding:** memory should preserve the conditions under which a lesson applies. Turning those conditions into checks in the current workflow can expose failures that separately passing examples miss.

Table 5: Behavioral studies of memory use. (a) Identical task, history, tools, and repair budget; only the added checks differ. (b) Timezone checks under different memory contents. Regressions count checks passed without memory but failed with it. Entries count checks, not independent tasks.

(a) Separate and composed checks on Django12663

Measured behavior	Current feedback	Separate history	Composed conditions
Official verification	Pass	Pass	Pass
Supplied historical examples	0/2	2/2	2/2
Withheld joint queries	2/3	1/3	3/3

(b) Requirement and implementation memory across 72 timezone combinations

Memory content	Passed	Failed	Regressions
Requirement	72/72	0	0
Implementation	40/72	32	16
Both	72/72	0	0

6 CONCLUSION

ContextGraph remembers when a lesson applies and what behavior it should preserve, linking transferable strategies and warnings to their source experience. Agents use their full context to select guidance before planning and after errors, then contribute new experience after each task. With DeepSeek V4 Pro, it achieves 69.2%, 35.1%, and 72.1% resolution on Verified500, Related-Lite99, and DeepSWE113. Gemini 3.1 Pro reaches 88.9% pass@1 on Verified500, while LoLBench pass@1 improves from 5.0% to 15.0%. These results establish experience graphs as a strong starting point for self-improving coding-agent memory.

REFERENCES

- Silin Chen, Shaoxin Lin, Yuling Shi, Heng Lian, Xiaodong Gu, Longfei Yun, Dong Chen, Lin Cao, Jiyang Liu, Nu Xia, and Qianxiang Wang. SWE-Exp: Experience-driven software issue resolution, 2025. URL <https://arxiv.org/abs/2507.23361v2>.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory, 2025. URL <https://arxiv.org/abs/2504.19413>.
- Soham Daga, Sreeram Sreedhar, and Dhruva Shah. Supermemory is state of the art in agent memory, 2026. URL <https://supermemory.ai/research/longmembench/>. Supermemory Research, LongMemEval-S evaluation report, May 2026.
- Tao Feng, Chongrui Ye, Tianyang Luo, Jingjun Xu, Xueqiang Xu, Haozhen Zhang, Zhigang Hua, Yan Xie, Shuang Yang, Ge Liu, and Jiaxuan You. ExpGraph: Model-agnostic experience learning with graph-structured memory for LLM agents, 2026. URL <https://arxiv.org/abs/2605.30712v1>.
- Shuzheng Gao, Wenhao Zeng, Zhaojian Yu, Jianqiao Wangni, Chaozheng Wang, Kai Cai, Shilin He, and Michael R. Lyu. SWE-Mem: Learning adaptive memory management for long-horizon coding agents, 2026. URL <https://arxiv.org/abs/2606.28434v1>. Version 1.
- Xuehang Guo, Zora Zhiruo Wang, Qingyun Wang, Graham Neubig, and Xingyao Wang. Enhancing software engineering through closed-loop memory optimization, 2026. URL <https://arxiv.org/abs/2606.05646v1>.
- Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/6ddc001d07ca4f319af96a3024f6dbd1-Abstract.html.
- Wenqi Huang, Charley Lee, Leonard Tng, and Serena Ge. DeepSWE: Measuring frontier coding agents on original, long-horizon engineering tasks, 2026. URL <https://arxiv.org/abs/2607.07946>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024. URL <https://arxiv.org/abs/2310.06770>.
- Kangsan Kim, Minki Kang, Taeil Kim, Yanlai Yang, Mengye Ren, and Sung Ju Hwang. Memory transfer learning: How memories are transferred across domains in coding agents, 2026. URL <https://arxiv.org/abs/2604.14004v1>. Version 1.
- LangChain. LangMem, 2025. URL <https://github.com/langchain-ai/langmem>. Software library.
- Yining Liu, Chenyu Yang, Boyan Li, Rui Mao, and Yuyu Luo. MIRA: Evidence-verified repair memory for text-to-sql correction, 2026. URL <https://arxiv.org/abs/2608.06950v1>.
- lolbench26. LoLBench: Long-horizon tasks on large software systems. Hugging Face dataset, 2026. URL <https://huggingface.co/datasets/lolbench26/LoLBench>.
- Z. C. Luo, J. C. Guo, W. J. He, S. Y. Wang, J. C. Yu, F. M. Zhao, Y. Chen, T. Cao, L. Q. Liu, N. Zheng, W. Xu, J. Jiang, and Z. M. Zhao. AdaRepair-Mem: Adaptive experience orchestration for repository-level program repair, 2026. URL <https://arxiv.org/abs/2609.20130v1>.
- Fangwen Mu, Junjie Wang, Lin Shi, Song Wang, Shoubin Li, and Qing Wang. ExpeRepair: Dual-memory enhanced LLM-based repository-level program repair, 2026. URL <https://arxiv.org/abs/2506.10484v2>. Version 2; accepted at FSE 2026.

- Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery, 2025. URL <https://arxiv.org/abs/2506.13131>.
- Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. ReasoningBank: Scaling agent self-evolving with reasoning memory. In *International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=jL7fwchScm>.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems, 2023. URL <https://arxiv.org/abs/2310.08560>.
- Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory, 2025. URL <https://arxiv.org/abs/2501.13956>.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html.
- SWE-agent Team. mini-SWE-agent: The minimal software engineering agent. Software repository, 2025. URL <https://github.com/SWE-agent/mini-swe-agent>.
- Haibo Wang, Zhuolin Xu, and Shin Hwei Tan. Testing refactoring engine via historical bug report driven llm, 2025. URL <https://arxiv.org/abs/2501.09879v2>.
- Qihao Wang, Ziming Cheng, Shuo Zhang, Fan Liu, Rui Xu, et al. MemGovern: Enhancing code agents through learning from governed human experiences, 2026. URL <https://arxiv.org/abs/2601.06789v2>.
- Zihan Wu, Jie Xu, Yun Peng, Chun Yong Chong, and Xiaohua Jia. MulVul: Retrieval-augmented multi-agent code vulnerability detection via cross-model prompt evolution. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8660–8673, 2026. doi: 10.18653/v1/2026.acl-long.391. URL <https://aclanthology.org/2026.acl-long.391/>.
- Peng Xia, Rujun Han, Zifeng Wang, Yanfei Chen, Yufan Zhang, Yoonho Lee, Chengsong Huang, Han Yu, Zhongying CuiZhu, Yifei Ming, Huaxiu Yao, Burak Gokturk, Tomas Pfister, and Chen-Yu Lee. RRSI: Regularized recursive self-improvement of agent harnesses, 2026. URL <https://arxiv.org/abs/2609.24972v1>. Version 1.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-Mem: Agentic memory for LLM agents. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/19909c36f51abc4856b4560aff3d36d6-Abstract-Conference.html.
- Yisen Xu, Jiayuan Zhou, Ruiqi Pan, and Tse-Hsun Chen. Reusing past repairs through hierarchical trajectory abstraction for coding agents, 2026. URL <https://arxiv.org/abs/2607.29658v1>. Version 1.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37, 2024. doi: 10.52202/079017-1601. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html.

- Xunjian Yin, Xinyi Wang, Liangming Pan, Li Lin, Xiaojun Wan, and William Yang Wang. Gödel Agent: A self-referential agent framework for recursively self-improvement. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 27890–27913, 2025. doi: 10.18653/v1/2025.acl-long.1354. URL <https://aclanthology.org/2025.acl-long.1354/>.
- Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin Gödel Machine: Open-ended evolution of self-improving agents. In *International Conference on Learning Representations*, 2026a. URL <https://openreview.net/forum?id=pUpzQZTvGY>.
- Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models. In *International Conference on Learning Representations*, 2026b. URL <https://arxiv.org/abs/2510.04618v3>.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, 2024. doi: 10.1609/aaai.v38i17.29936. URL <https://ojs.aaai.org/index.php/AAAI/article/view/29936>.
- Yujun Zhou, Kehan Guo, Haomin Zhuang, Xiangqi Wang, Yue Huang, Zhenwen Liang, Pin-Yu Chen, Tian Gao, Nuno Moniz, Nitesh V. Chawla, and Xiangliang Zhang. Getting better at working with you: Compiling user corrections into runtime enforcement for coding agents, 2026. URL <https://arxiv.org/abs/2606.13174v1>.
- Jiayuan Zhu, Junde Wu, Minhao Hu, Shengda Zhu, Jiazhen Pan, Weixiang Shen, Yijun Yang, Fenglin Liu, Jianye Hao, Yueming Jin, Qirong Ho, and Min Xu. Swe context bench: A benchmark for context learning in coding, 2026. URL <https://arxiv.org/abs/2602.08316v3>. Version 3.

A MAIN BENCHMARK COMPARISONS

Table 6: Repair resolution rates (%). Panel A tabulates Figure 4; the LoLBench comparison is in Table 12. Panel B groups published reference results by their reported configuration. Bold marks the best result per benchmark in Panel A. An em dash indicates that the cited source does not report the corresponding benchmark.

A. Three-benchmark comparison

Method	Verified500	Related-Lite99	DeepSWE113
No memory	61.6	30.6	60.8
FAISS Flat	65.2	26.8	64.6
ExpeL	62.4	20.7	58.7
ExpeRepair [†]	63.6	31.3	64.4
ReasoningBank	67.0	24.9	61.7
ACE [†]	62.2	28.1	62.8
Supermemory	63.6	26.5	66.2
ContextGraph	69.2	35.1	72.1

Verified500, Related-Lite99, and DeepSWE113 use DeepSeek V4 Pro 0813 and mini-SWE-agent. Related-Lite99 and DeepSWE113 rates are means across five runs, each reset to the same initial memory pool, with tasks evaluated in issue creation-time order. ContextGraph adds lessons from completed successful and failed attempts to memory for subsequent retrieval; baselines use their open-source defaults for memory updates. The initial Verified500 memory draws on Nebius SWE-bench Verified trajectories; the DeepSWE113 memory draws on trajectories from LoLBench repositories. [†] ExpeRepair and ACE are our adaptations to mini-SWE-agent. Method sources: ExpeL (Zhao et al., 2024), ExpeRepair (Mu et al., 2026), ReasoningBank (Ouyang et al., 2026), ACE (Zhang et al., 2026b), and Supermemory (Daga et al., 2026).

B. Published reference results, grouped by their reported configuration

Method	Backbone	Verified500 (%)	Related-Lite99 (%)	Source
ExpeRepair	Claude 3.5 Sonnet	57.2	—	ER
ExpeRepair	Claude 4 Sonnet	74.6	—	ER
ReasoningBank	Gemini 2.5 Flash	38.8	—	RB
ReasoningBank	Gemini 2.5 Pro	57.4	—	RB
No memory	Claude Sonnet 4.5	—	26.26	SC
Supermemory	Claude Sonnet 4.5	—	30.30	SC
Oracle summary	Claude Sonnet 4.5	—	34.34	SC

ER: ExpeRepair, Table 1; RB: ReasoningBank, Table 2; SC: SWE-ContextBench, Table 4. ER uses its own repair pipeline with occasional o4-mini fallback; RB uses mini-SWE-agent; SC uses Claude Code. Oracle summaries supply the known related history. Published scores use their original budgets and memory settings.

B ADDITIONAL SYSTEM COMPARISONS

These comparisons examine summary and episodic memory across models and attempt budgets. Resolution rates are percentages; gains marked pp are percentage-point differences from no memory.

Memory access and evaluation. The agent uses its full current context to search the experience pool, read records, and follow links to relevant strategies. The Related-Lite99 summary comparison uses 300 summaries and one attempt per task. The pool-size study varies this initial pool from 50 to 300 summaries (Table 3). MiniMax M2.7 uses an API compatibility layer. The curated-episode comparison uses 291 repair records with prior code history, reference fixes, and verification hints. Official test results are withheld from the agent throughout, including between attempts.

Source-similarity sensitivity. A separate analysis removes ten targets whose source fixes have token-Jaccard similarity of at least 0.85. These are distinct tasks, not target trajectories in the memory pool. On the remaining 89 tasks, DeepSeek V4 Pro resolves 21.35% with memory versus 14.61% without memory ($p = .031$).

Table 7: Curated-episode Related-Lite99 replication, one pass per backbone. Resolution rates are percentages. The last column gives no-memory/ContextGraph rates on the fixed 89-task subset after removing ten near-duplicate targets, followed by the paired p value. N/C only counts tasks resolved exclusively by no memory/ContextGraph.

Backbone	No memory	ContextGraph	N/C only	p	89-task subset
DeepSeek V4 Pro	14.14	21.21	0/7	.016	14.61/21.35 ($p = .031$)
DeepSeek V3.2 exp	16.33	20.20	4/8	.39	16.85/20.22 ($p = .55$)
GLM 5.2	17.17	24.24	1/8	.039	16.85/23.60 ($p = .070$)
Kimi K3	29.29	31.31	3/5	.73	29.21/31.46 ($p = .73$)

Table 8: Separate 50-task development comparison; tasks are disjoint from the test sets. Completion uses all 50 tasks; resolution uses completed evaluations. The recorded 66.6% retains its original display precision.

Method	Completion (%)	Resolution (%)
ContextGraph	96.0	66.6
ExpeL	100.0	64.0
FAISS Flat	98.0	61.2
No memory	100.0	58.0

Table 9: Experience-graph snapshot, using text-embedding-3-large vectors with 3,072 dimensions. The executable-example bank for behavioral studies uses Qwen embeddings.

Node label	Count	Role
Trajectory	1,795	Source episode
Fragment	13,813	Action and observation slice
ErrorPattern	586	Error type and keywords
Strategy	9,600	Extracted rule
CanonicalRule	7,835	Consolidated rule
PlaybookEntry	8,910	Retrieval-ready entry
Community	781	Linked-fragment summary
ProblemSummary	1,795	Episode problem abstraction
Total nodes	45,115	
Total edges	73,975	Seven typed relations

C ADDITIONAL DEVELOPMENT COMPARISONS

Fail-to-pass tests check the requested fix; pass-to-pass tests check behavior that worked before the repair.

Table 10: Django14404 development comparison. All candidates pass the official 2 fail-to-pass and 323 pass-to-pass tests.

Candidate	Prefix	Query	Prefix + query
Initial repair	Pass	Fail	Fail
Issue-based revision	Pass	Fail	Fail
Historical patch revision	Pass	Pass	Pass
Executed relation revision	Pass	Pass	Pass

Intervention scope. Content comparisons start fresh agents from the original development target; refinement comparisons start from a common completed candidate. Additional MySQL, named-zone, and ordinary-subclass checks were constructed after inspecting the corresponding repairs, then applied across their comparison group. The three-arm composition comparison holds history, the current example and failure, tools, and instructions fixed. Two arms additionally receive either composed checks or separate historical checks, as shown in Table 15.

Operation-dependent preservation. In Matplotlib20826, clearing and replotting twin axes restores finite data bounds, so failure of an isolated historical plotting example does not describe the state after that workflow. In Django16819, canceling a same-name index remove/add pair loses a required change of indexed column. These observations motivate applying memory conditions along the current operation sequence.

Additional development outcomes. Two preliminary baseline runs remain unresolved. The DeepSWE Kea task passes 9/12 fail-to-pass and 139/139 pass-to-pass tests. The JDK5 LoLBench task initially submitted no patch; recovering its recorded patch without another model call gives 0/5 fail-to-pass and 51/51 pass-to-pass tests. Both receive a task-level reward of zero.

Executable-example coverage. The 273-source development bank provides at least one checked source from the same repository for 133/150 Verified targets, 91/99 Related-Lite targets, 2/100 LoL-Bench targets, and 0/113 DeepSWE targets. These counts describe the executable-example bank used for the behavioral studies. Its records pair a historical requirement with an executable check.

D ADDITIONAL BENCHMARK AND AGENT RESULTS

This appendix gives the DeepSeek V4 Pro summary-memory results plotted in Figure 5, the LoLBench comparison from Section 5.2, and token measurements. Gemini results appear in Table 2. The three-benchmark comparison is tabulated in Table 6; additional comparisons appear in Appendix B.

Table 11: Related-Lite99 summary memory with DeepSeek V4 Pro, one attempt per task and a 300-summary pool, matching Figure 5. Bold marks the best automatic memory result; the oracle receives a known related summary.

Method	Resolution rate (%)
No memory	30.6
Random summaries	27.1
Mem0	26.8
LangMem	34.0
ContextGraph	35.1
Oracle summary	27.8

Token efficiency. On SWE-bench Verified500, the recorded comparison uses 236K tokens per task without memory and 201K with ContextGraph, a 14.8% reduction (approximately 15%).

LoLBench Python subset. We select 20 Python tasks from LoLBench (lolbench26, 2026), which evaluates long-horizon development from enhancement proposals in large software systems. The agent is Codex with GPT-5.5. The initial experience graph is built from other Python tasks, separate from the 20 evaluation tasks. During evaluation, ContextGraph adds lessons from completed attempts, including successes and failures, for subsequent retrieval. Baseline memory updates follow each method’s open-source defaults.

Table 12: Success rates on 20 LoLBench Python tasks with Codex and GPT-5.5, as reported in Section 5.2. $\text{pass}@k$ reports the fraction of tasks solved within k attempts.

Method	Metric	Success rate (%)
No memory	pass@1	5.0
FAISS Flat	pass@1	5.0
ExpeL	pass@1	5.0
ExpeRepair	pass@1	5.0
ReasoningBank	pass@1	5.0
ACE	pass@1	5.0
Supermemory	pass@1	5.0
ContextGraph	pass@1	15.0
ContextGraph	pass@3	25.0

E FULL-CONTEXT AGENT-DIRECTED MEMORY SELECTION

At step t , the agent uses the complete context C_t : the issue and conversation, inspected code, edits, actions, and observed failures. It can search the whole experience pool, read source records, follow links between strategies and errors, and compare lessons across repositories. Its selection follows Equation 2: the agent decides which records to inspect and retain, without a fixed relevance score or number of nearest neighbours. Each search or read adds its results to H_j before the next memory action; the selected lessons form $\mathcal{M}_t$.

The selected memory contains strategies and warnings, their source context, and the conditions under which they are useful. Before planning, selection helps choose an investigation strategy. After an error, the updated context helps revise the diagnosis and find additional guidance. After each completed task, the graph incorporates new experience for subsequent tasks. The pool therefore evolves along the issue-ordered task stream; official benchmark test results are never part of the agent’s context or stored memory.

Algorithm 1 ContextGraph across a stream of coding tasks

Input: initial trajectories H , coding tasks $q_1, \dots, q_T$.

Output: candidate patches and an updated experience graph G .

1. Extract strategies and warnings from H ; build G and its retrieval entries.
 2. **For each** task q_t in issue creation-time order:
 - a. Use the full task context to search G and select guidance for planning.
 - b. Execute investigation, editing, and testing actions; record the trajectory.
 - c. After an error, search again with the updated full context; revise the plan.
 - d. Continue the repair loop until the task ends or its budget is exhausted.
 - e. Distill experience from the trajectory and agent-visible outcomes.
 - f. Update G and retrieval entries; expose the new experience to later tasks.
-

F BEHAVIORAL STUDIES OF RETRIEVED GUIDANCE

The Django example in Section 3 illustrates strategy transfer. Here we examine how agents apply a retrieved behavioral requirement: what context they need, which interactions they miss, and how feedback changes the next edit. Figures 2 and 6 show the checks and workflow used for these studies.

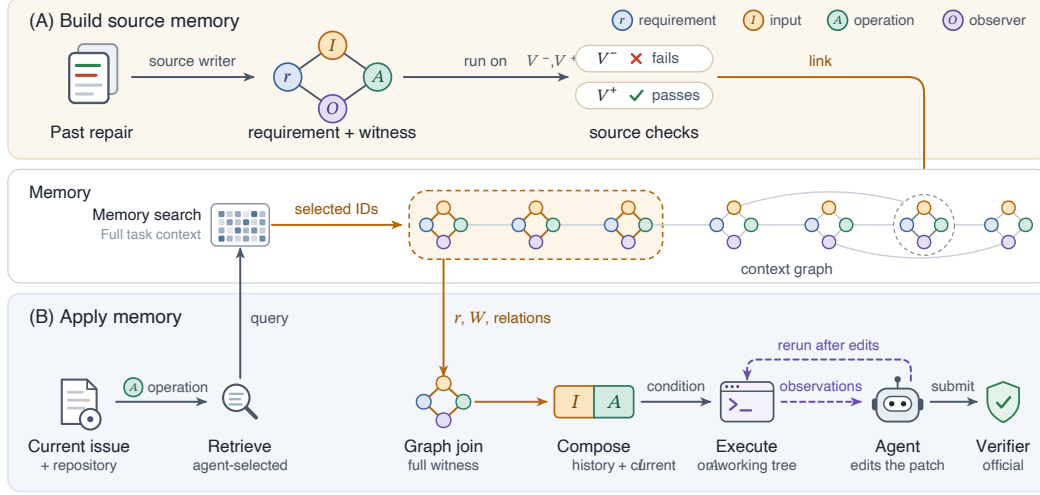


Figure 6: Workflow for the behavioral studies. (A) Historical repairs supply requirements and checked examples linked in memory. (B) Retrieval selects relevant sources; graph relations support composition with the current operation. Execution feedback guides edits before verification.

Retain the context that makes advice useful. In four paired runs, memory helps preserve query strings, remove redundant uniqueness while retaining primary keys, honor destination timezones, and preserve named-tuple ranges (Table 13). Both repairs in the first three pairs pass official verification.

Table 13: Joint current/historical behavior in complete-agent comparisons. The columns report whether repairs with and without memory preserve both the current behavior and the historical requirement.

Target	Historical addition	No memory	Memory
Django14404	Preserve query string	Fail	Pass
Django12708	Delete redundant uniqueness	Fail	Pass
Django11138	Honor destination timezone	Fail	Pass
Django12663	Preserve named-tuple range	Fail	Pass

We then vary the memory supplied for the constraint and timezone tasks: a written requirement, implementation details, both, or neither (Table 14). All eight repairs pass official verification. For constraints, the requirement alone leads the agent to test different columns and miss the same-column bug; the implementation retains that condition. For timezones, the requirement directs the agent to a suitable helper, whereas the old implementation leads to errors on named timezones.

On Django12663, both retrieved memory sets contain the relevant named-tuple source, yet both initial repairs miss its behavior. Preserving its constructor convention fixes named tuples while keeping ordinary subclasses working; passing every sequence’s elements as separate arguments breaks the latter (Appendix Figure 7). A candidate-selection diagnostic also shows that an exact operation filter discards this useful source (Appendix G.1). **Finding:** advice needs its applicability context: shared columns and constructor conventions determine how these lessons transfer.

Table 14: Memory-content interventions: one fresh attempt per input and development task. All eight pass official verification. MySQL and named-zone checks were constructed after inspecting the timezone repairs and applied to all four submissions.

Memory input	Constraint joint	Timezone joint	MySQL cases	Named-zone cases
None	Fail	Fail	Pass	Fail
Requirement	Fail	Pass	Pass	Pass
Implementation	Pass	Pass	Pass	Fail
Both	Pass	Pass	Fail	Pass

Check a historical requirement in the current workflow. A Django12663 repair passes the current lazy-User query and historical named-tuple example separately, but fails their combination. The range adapter inserts the historical type into the current query, retaining the lazy User and nested query (Figure 2).

Three agents start from the same unrepaired code, history, current program, tools, and instructions (Table 15). Separate historical checks recover both historical examples but only one of three withheld joint queries. Composed checks recover both historical examples and all three joint queries, while preserving ordinary subclasses.

Table 15: Django12663 repairs from original code with identical history, current program and failure, tools, and instructions. Two arms add composed or separate historical checks. Historical examples are supplied to the corresponding arms; joint queries and ordinary subclasses are withheld. Mechanical checks probe input representations. Rows count checks, not independent tasks.

Measured behavior	Current feedback	+ Separate history	+ Composed conditions
Official target verification	Pass	Pass	Pass
Supplied historical named-tuple examples	0/2	2/2	2/2
Withheld nonempty joint queries	2/3	1/3	3/3
Withheld ordinary subclasses	2/2	2/2	2/2
Mechanical scalar / named tuple / tuple	2/3	2/3	3/3

Timezone composition varies the database, application, and output zones. Requirement and Both pass all 72 combinations; Implementation fails 32, including 16 passed without memory. **Finding:** combining a historical trigger with the current operation exposes interactions that separate checks miss.

Use failure observations to guide the next edit. Two fresh agents revisit the failed constraint repair with the same requirement. Textual review fixes the primary-key case but misses the ordinary unique-field case; adding the historical check and its failures helps the agent distinguish constraint names and fix both. On the timezone task, further issue-based review leaves the patch unchanged, while the historical failure leads to a revision honoring the destination timezone.

Of four fresh refinements of the Django12663 candidate, history alone passes all seven measured conditions in one of two attempts; the joint program and failure do so in two of two. All four pass official verification. For the simpler redirect task, a requirement sentence already yields the same correct implementation as requirement-plus-feedback. **Finding:** targeted failure feedback helps agents act on a known requirement when textual review alone leaves the defect unresolved.

G DEVELOPMENT CASES AND INPUT CONSTRUCTIONS

This appendix gives the inputs, source requirements, and repair observations for the behavioral studies in Appendix F.

G.1 RETAIN THE CONTEXT THAT MAKES ADVICE USEFUL

The behavioral studies use development tasks disjoint from the benchmark test sets. They vary the information supplied to DeepSeek V4 Pro, at temperature zero and matched budgets within each comparison. They measure whether historical guidance helps preserve a required behavior in the submitted repair. Table 13 presents four such additions. Django14404 uses the selected source issue and patch; Django12708 and Django11138 come from a fixed six-task group receiving selected rules as plain text per task. Both conditions pass official verification on all six tasks. The historical requirements distinguish the completeness of those repairs.

An omitted behavior of the same API. Django14404 reports that administrative redirects lose a deployment prefix. Both agents fix the prefix and pass official verification, but only the repair informed by source16612 also preserves the query string. A joint input combines the prefix with repeated and percent-encoded query values, `keep=a%2Fb&keep=c%20d`. Only the history-informed repair preserves both.

A neighboring operation. Django12708 reports ambiguous deletion of an ordinary index when a unique constraint uses the same columns. The selected source, 15732, instead concerns deletion of a unique constraint sharing its column with a primary key. Both repairs fix ordinary-index selection; only the memory-informed repair also excludes primary keys when deleting unique constraints. The source-generated PostgreSQL check runs unchanged on both target repairs. A separate combined input creates an ordinary index, redundant uniqueness, and a primary key on the same column. Both repairs preserve ordinary-index deletion, but only the memory-informed repair also deletes redundant uniqueness while retaining the primary key and data. History supplies a requirement for the neighboring operation.

An independent argument of the same operation. Django11138 requires date conversions to respect the database’s source timezone. Source13363 requires `TruncDate` and `TruncTime` to honor an expression’s explicit destination timezone. Both agents repair the source timezone, but only the memory-informed repair also honors the destination. A source-generated check distinguishes the historical versions and runs unchanged on the target repairs. On an independent SQLite input combining a UTC+1 database with an explicit New York destination, only the memory-informed repair returns the correct date and time. The issue and history constrain different ends of the conversion.

A historical input type in the current query. A fourth pair repairs Django12663 from its original code, comparing no memory with ContextGraph using source13590. Both agents receive the same public current program, tools, backbone, and repair budget; only the memory arm receives the historical requirement and its checks. Each arm completes one repair. The joint checks combine the current lazy User with a named-tuple range in nonempty queries. The no-memory repair fails these checks, while the memory-informed repair passes.

A requirement can suffice without the source implementation. On the redirect task, retrieved text memory and its original issue/patch replacement produce identical repairs preserving the joint behavior. Two new agents then revise the ordinary patch with a source-generated requirement: the redirect must include the original query string. One also receives the check and its failed observations. Both submit identical repairs passing official and joint checks. The sentence-only agent finds `get_full_path()` in the current repository: memory supplies the behavior, and the agent locates its implementation.

Which information makes the requirement usable? We select the constraint and timezone benefit cases and run four fresh complete agents per task, fixing the single historical source and its delivery before editing. Using only the historical task, we construct a written requirement and an implementation reference; the combined input concatenates them unchanged. All eight repairs pass official verification (Table 14). On Django12708, only Implementation and Both also remove redundant uniqueness while preserving the primary key. The Requirement agent tests a field with `unique=True`, but places `unique_together` on a different, two-column set. This nearby passing test loses the same-column condition stated in memory. On Django11138, all three memory inputs preserve the explicit destination timezone, while the fresh no-memory repair omits it. The

				ADDITIONAL DIAGNOSTIC
	Official verification	Historical named tuples (2)	Named tuple + lazy User (1)	Ordinary tuple/list subclasses (2)
Memory supplied for revision				
Original issue + patch	Pass	Pass	Pass	Pass
Requirement sentence	Pass	Fail	Fail	Pass
Requirement + check + execution failures	Pass	Pass	Pass	Fail

Figure 7: Memory content changes which behaviors a repair preserves. One revision per input starts from the same Django12663 candidate and source13590. Parentheses give case counts; each cell reports all cases in its group. The ordinary-subclass inputs were chosen after inspecting the feedback repair. All three repairs pass official verification, while only original history passes every displayed behavior.

Requirement agent finds `self.get_tzname()` in the target implementation without being given that helper in memory.

The timezone repairs also differ on further conditions. The repair given Both still generates an unnecessary MySQL conversion between identical zones. The repair given Implementation instead misinterprets a named Paris source timezone, producing New York time 19:21 instead of 17:30 on a summer input. The repair given Requirement passes both additional diagnostics. These concrete omissions distinguish carrying a requirement, applying its triggering condition, and implementing the resulting behavior correctly. Two subsequent agents start from the original unrepaired code and receive the same constraint requirement, with or without an example validated on the historical repair; both pass official and joint checks. The same requirement succeeds when its trigger is reconstructed correctly. Preserving this input context helps explain when the advice is useful.

Preserve the input relation. In a candidate-selection diagnostic on Django12663, both a set of three code-connected sources and the ten most similar sources contain the historical named-tuple repair, source13590, yet neither initial repair recovers its behavior. Three fresh revisions start from the same candidate and source (Figure 7). Original history and execution feedback recover named tuples; the requirement sentence alone does not. The feedback repair expands every non-builtin sequence into positional arguments, which also breaks ordinary tuple and list subclasses. Original history retains the constructor distinction and passes all displayed behaviors. For this behavioral study, the useful context includes how the input type is constructed, alongside its endpoint values.

Retain candidates that can supply an omitted condition. In the candidate-selection diagnostic, an exact operation filter excludes source13590 despite its first-place FAISS similarity rank. Retaining this source allows its stored input relation to support the joint check described next. The diagnostic illustrates why useful guidance can come from a related operation. In the agent-directed workflow (Figure 6), the agent selects source records and their input relations support composition.

G.2 CHECK A HISTORICAL REQUIREMENT IN THE CURRENT WORKFLOW

Join the historical input to the current operation. The current issue involves a lazy User in a nested query; history contributes a named-tuple range. On a separate history-informed repair, the current scalar query and the historical range query each pass, while a query combining them fails. The combined check reveals that the patch handles one reconstruction of the input container but misses another later in the query-processing code.

The deterministic adapter in Equation 5 reaches the same omission while retaining the public program’s lazy User and nested annotation. It produces a named-tuple point range and an ordinary-tuple neighbor. Retrieving the input relation from the graph lets the adapter reconstruct the same check used in this repair comparison.

Three matched agents compare current, composed, and separate checks. Three agents start from the original unrepaired code with identical history, current scalar program and failure, tools, and instructions to run the supplied checks before editing and submission. The current-feedback arm receives this shared input. The composed arm additionally receives the two composed programs and their failures; the separate-history arm receives standalone historical checks. All three repairs pass official verification (Table 15). The repair given only current feedback fails both historical examples and one of three withheld joint queries. The repair given composed checks passes both historical examples and all three joint queries, while preserving the ordinary subclasses. The agent given joint checks runs the joint input before editing and again after removing a global conversion that broke an existing test. Its remaining range-endpoint failure directs the final correction.

Separate checks recover history but miss the interaction. The separate-history arm uses standalone named-tuple and ordinary-tuple checks with their original objects. Check count, feedback rounds, execution instructions, and repair budget match the composed arm. This repair passes official verification, both historical examples, and both ordinary-subclass checks, but only 1/3 joint queries and 2/3 mechanical checks; composed conditions yield 3/3 in each group (Table 15).

Compose roles to expose a broader requirement. The timezone relation provides another example. A fixed adapter varies the historical example’s three zones across database, active application, and destination roles and shifts the input by 182 days. Of 72 combinations, the source patch changes 20 from fail to pass; 20 pass on both versions and 32 fail on both. Applying the family to the four timezone content interventions shows that Requirement and Both pass all 72. Implementation passes the 40 source-supported combinations but fails the remaining 32, including 16 passed without memory. Role composition exposes how the current source-timezone requirement interacts with the historical destination requirement. A complete repair can therefore satisfy combinations beyond those corrected by the historical implementation.

G.3 USE FAILURE OBSERVATIONS TO GUIDE THE NEXT EDIT

The addition can guide further repair. Two fresh agents start from the same completed ordinary timezone patch. Issue-based additional review returns it unchanged; supplying a generated historical check and its failed observations leads the other agent to use `self.get_tzname()` in both truncation methods. Both revisions pass official verification, but only the latter passes the historical and joint inputs. The intervention supplies corrective information that further issue-based review does not recover in this comparison.

Execution feedback makes an omitted condition concrete. Two fresh agents refine the same original failed Requirement candidate, both receiving its requirement again. One also receives the unchanged source-generated check and its actual failures for a field with a primary key or `unique=True`. Requirement-only review excludes the primary key but still fails the ordinary unique-field condition. Feedback adds generated-name disambiguation and passes both conditions. Both revisions pass official verification and the withheld joint sequence. The added behavior is recovery of the ordinary unique-field condition identified by feedback.

Feedback from the composed query. The Django12663 example also provides a refinement comparison. Four fresh refinements of its history-informed candidate illustrate the use of feedback: history alone yields one of two repairs passing all seven measured conditions; adding the constructed joint program and its failed observation yields two of two. All four pass official verification. Both agents given joint feedback execute the joint input and repair the later container reconstruction.

H BEHAVIORAL ANALYSIS WITH EXECUTABLE CONDITIONS

The behavioral studies augment retrieved guidance with task-specific programs to examine whether a repair preserves the intended behavior. The programs make requirements testable in these cases; the main memory interface returns strategies, warnings, and source context. Figures 2 and 6 illustrate the comparison and workflow; this appendix gives the construction protocol.

Values alone do not describe a condition. In Python, a named tuple is a tuple whose fields also have names. In our examples, constructing it takes two arguments; an ordinary tuple subclass takes one iterable. Both can hold the same values but exercise different code. Likewise, a timezone can describe the stored data or the requested output. These checks depend on the input roles as well as their values.

Two passing examples can hide a failing combination. Our Django case uses a lazy User object: a wrapper that loads the user when needed. A query using this object passes, as does an older range query using a named tuple to hold its endpoints. Putting the lazy object inside the named-tuple range in the current query fails. This motivates *condition composition*: inserting a historical trigger into the current operation (Figure 2).

H.1 EXECUTABLE-CONDITION STUDY PROTOCOL

The executable-analysis extension connects example construction to a repair loop (Figure 6). Construction records what a past repair taught and which conditions make that lesson relevant. During repair, retrieval selects useful experience, composition brings its conditions into the current workflow, and execution turns them into feedback for the next edit. Stored links connect a retrieved requirement to its supporting example and expected behavior.

H.2 RETAIN THE CONTEXT THAT MAKES ADVICE USEFUL

For these studies, an analysis record augments a retrieved requirement with a test example and its results before and after the historical repair:

$$m = (r, W, E^-, E^+), \quad W = (I, A, O), \quad (4)$$

where r states the requirement, I constructs its triggering input, A performs the operation, and O observes the expected outcome. We call W a witness: an executable example that fails on the historical code V^- and passes on its repaired version V^+ . E^- and E^+ record these executions.

A language-model writer extracts the requirement and example from the historical issue, patch, and relevant code. The example is checked against the historical repair, then stored with its requirement and execution results. The graph also records how inputs participate in an operation: which values form a range, for example, or which timezone specifies the output. These relations preserve the information needed to adapt an experience after retrieval.

Retrieve the requirement together with its context. The study retrieves relevant historical repairs, then follows record links to return their requirements, programs, execution results, and input roles together. A useful source may concern a related operation: a fix for deleting one kind of database constraint can help repair deletion of another kind. Keeping the requirement connected to its context gives the agent both a lesson and the information needed to apply it. The agent selects records using its full context; Appendix I gives the study settings.

H.3 CHECK A HISTORICAL REQUIREMENT IN THE CURRENT WORKFLOW

Composition turns retrieved experience into a check for the current task. It starts with P_q , a runnable program prepared from the current issue, and a historical requirement applicable to the current code. For these studies, researchers prepare the setup and observation code. An *adapter*, a small program-transformation routine, uses the stored input relations to insert the historical trigger into P_q . The check keeps the current workflow and tests the expected behavior under this added condition.

For example, a historical range type can be introduced into the current query while preserving its input objects and query structure (Figure 2). Running this combination exposes interactions that neither the original query nor the historical example exercises alone. The same design applies to timezone conversion: the historical destination requirement is checked together with the current source-timezone behavior. Appendix I gives the input preparation and adapter constructions.

H.4 USE FAILURE OBSERVATIONS TO GUIDE THE NEXT EDIT

The agent runs the constructed checks and uses failures to revise the patch. Each failure identifies an input and expected behavior that the repair needs to satisfy. Rerunning the same check gives successive edits a concrete repair objective.

I BEHAVIORAL-STUDY IMPLEMENTATION AND EVALUATION DETAILS

This appendix explains how the behavioral studies construct and use executable checks (Appendix H). Section 4 describes the strategy memory used in planning and error recovery.

I.1 CONSTRUCTING HISTORICAL EXAMPLES

A language-model writer reads the historical issue, patch, and relevant code to propose a requirement and runnable example. Execution on the historical versions and researcher review check that the example exercises the intended behavior. Accepted examples are stored with their requirement, input roles, program, and before/after execution results.

I.2 RETRIEVAL AND REPAIR SETTINGS

The agent uses its full current context to select applicable source records from the experience pool. Search and graph links help locate requirements and their validated examples; complete records preserve input types and argument roles for the subsequent checks.

Within each comparison, agents share the target code, backbone, tools, instructions, and repair budget. The comparison changes the supplied memory or feedback, as specified in Appendix G.

I.3 PREPARING THE CURRENT EXECUTABLE PROGRAM

The development range example extracts model and test snippets from the public issue and adds researcher-written setup and observation code. The range and timezone adapters are also researcher-written. The case inputs, expected behaviors, and control conditions are given in Appendix G.

I.4 COMPOSITION INTERFACE

The composer implements Equation 3. It takes the current program P_q , memory m , input bindings b , and available adapters. A binding identifies which object plays each role, together with its type, constructor, argument position, and code location. The composer returns the check, code that inspects its outcome (the observer), the mapping used, and whether construction succeeded. The steps are:

1. Match an adapter to the current operation and its type requirements.
2. Map current objects to historical input roles, preserving both triggers and the current operation sequence.
3. Construct the program and check its fixture, bindings, and observer. The expected outcome comes from the requirement, independently of the candidate patch.
4. Execute the check and return a behavioral failure as repair feedback. Record setup or observation errors separately so they can be distinguished from failures of the requirement.

The current adapters cover two Django operation families. The range adapter transforms the public query’s syntax tree, changing an equality lookup for x into a range with both endpoints equal to x , using the historical named-tuple type R :

$$P_q(x) \mapsto P_q^{\text{range}}(R(x, x)). \quad (5)$$

When equality and this one-point range select the same normalized, non-null value, the expected rows stay unchanged. The adapter preserves the lazy User and nested query; replacing that object with a lazy integer would lose the interaction. An ordinary-tuple range checks preservation of existing behavior. The timezone adapter constructs combinations of database, active application, and

1242 destination roles. Extraction code recovers the bindings these adapters support. Adding an operation
1243 family requires another adapter and its development tests.
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295

J PUBLISHED SWE-CONTEXTBENCH REFERENCE RESULTS

Table 16 reproduces the resolved rates from Table 4 of Zhu et al. (2026). Supermemory leads the automatic memory settings reported there; oracle summaries use known task relationships to supply the relevant history. These published results provide context for our Related-Lite experiments. Direct comparisons require matching task IDs, backbone, agent, and attempt budget.

Table 16: Published SWE-ContextBench Lite results: 99 related tasks, Claude Sonnet 4.5. Rates are copied as reported, including OpenViking’s 29.20%. Δ is the percentage-point change from no context. Oracle rows are separated from automatic retrieval.

Setting	History access	Resolved (%)	Δ (pp)
No context	None	26.26	0.00
Free context	Agent-selected trajectories	26.26	0.00
Free summary	Agent-selected summaries	22.22	−4.04
Mem0	Automatic retrieval	24.24	−2.02
OpenViking	Automatic retrieval	29.20	+2.94
Supermemory	Automatic retrieval	30.30	+4.04
Oracle context	Supplied related trajectory	27.27	+1.01
Oracle summary	Supplied related summary	34.34	+8.08

On the separate 376-task set, Table 3 reports its best resolution rate of 23.94% for GPT-5.3 Codex with oracle summaries. Supermemory’s 55.95% in Table 4 is a test-level FAIL_TO_PASS rate, not task resolution. LangMem has retrieval-quality results in Table 5 but no resolved rate in Table 4.

LIMITATIONS

The main comparisons measure the full memory systems; their aggregate gains combine representation, retrieval, and updating. Behavioral cases explain specific repair decisions, while the 20-task LoLBench study covers one language and a small part of that benchmark. Broader task streams and controlled component comparisons would clarify when each design contributes most. Detailed settings and additional results accompany the appendices.

REPRODUCIBILITY STATEMENT

Code and experiment configurations will be provided through an anonymous repository link. The main text defines the memory workflow, benchmark settings, and reported metrics. The appendices provide numerical comparisons, behavioral-study inputs and checks, and supplementary implementation details. ContextGraph’s main comparisons include online experience updates; baseline updating follows the corresponding open-source implementation.

ETHICS STATEMENT

The reported evaluation uses software-engineering benchmarks. No confidential enterprise scenarios or data are included. Retrieved strategies and warnings guide the agent’s investigation; patches are evaluated through the benchmark harnesses.

AI USE STATEMENT

Generative AI assisted literature review, method development, implementation, experiments, analysis, and writing, including drafting and revising the abstract.